#define and Loops

define

- In C programming, **#define** is a preprocessor directive that is used to define macros.
- The macros are the identifiers defined by #define which are replaced by their value before compilation.
- We can define constants and functions like macros using #define.

Syntax: [with value]

#define MACRO_NAME *value*

Syntax [with expression]

#define MACRO_NAME *(expression within brackets)*

```c
#include <stdio.h>


#define PI 3.14159265359 // Defining macros with constant value

int main()
{

    int radius = 21;
    int area;

        area = PI * radius * radius; // Using macros to calculate area of circle


    printf("Area of Circle of radius %d: %d", radius, area);

    return 0;
}
```

‘continue’ statement

```
#include <stdio.h>

// Defining macros with expression
#define PI (22 / 7)

int main()
{

    int radius = 7;
    int area;

    // Using macros to calculate area of circle
    area = PI * radius * radius;

    printf("Area of Circle of radius %d: %d", radius, area);

    return 0;
}
```

Loops

Definition

Loops in programming are used to repeat a block of code until the specified condition is met.

A loop statement allows programmers to execute a statement or group of statements multiple times without repetition of code.

```
#include <stdio.h>
int main()
{
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");
   printf( "Hello World\n");

   return 0;
}
```

There are mainly two types of loops in C Programming:

- **Entry Controlled loops:** In Entry controlled loops the test condition is checked **before** entering the main body of the loop.

Eg: For Loop and While Loop is Entry-controlled loops.

- **Exit Controlled loops:** In Exit controlled loops the test condition is evaluated **at the end** of the loop body.
- The loop body will execute at least once, irrespective of whether the condition is true or false.

Eg: do-while Loop is Exit Controlled loop.

Loop Type	Description
for loop	first Initializes, then condition check, then executes the body and at last, the update is done.
while loop	first Initializes, then condition checks, and then executes the body, and updating can be inside the body.
do-while loop	do-while first executes the body and then the condition check is done.

While Loop

In the While loop, the execution is terminated on the basis of the test condition.

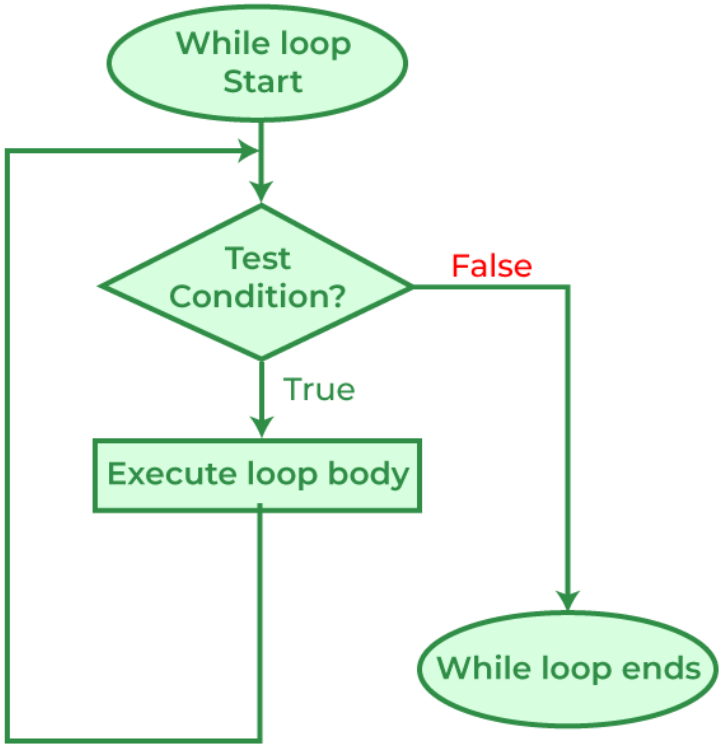
If the test condition is false then it will break from the while loop else body will be executed.

While loop does not depend upon the number of iterations.

Syntax:

```
initialization_expression;
while (test_expression)
{
    // body of the while loop update_expression;
}
```

Flow Diagram for while loop



While Loop

```
#include <stdio.h>
 int main()
{
  // Initialization expression
  int i = 1;
   // Test expression
  while(i < =10)
  {
    // loop body
    printf( "Hello World\n");
        // update expression
    i++;
  }

  return 0;
}
```

While loop - problems

1. WAP to print all the numbers below a specified number
2. WAP to print all the odd/even numbers below a range
3. Write a C program that prompts the user to enter a positive integer. Use a while loop to print the multiplication table for that number up to 10.
4. Write a C program that calculates the sum of n-natural numbers
5. Write a C program that prompts the user to input a series of integers until the user stops entering 0 using a while loop. Calculate and print the sum of all the positive integers entered.
6. Write a C program that prompts the user to enter a positive integer. It then calculates and prints the factorial of that number using a while loop.
7. Write a C program that implements a program to check if a given number is a palindrome using a while loop.
8. Write a C program to find and print the first 10 Fibonacci numbers using a while loop.

```
6. #include <stdio.h>
int main()
{
int number;
long factorial = 1;
printf("Input a positive integer: ");
scanf("%d", &number);
if (number < 0)
{
printf("Error: Input enter a non-negative integer.\n");
return 1;
while (number > 0)
{
    factorial *= number;
    number--;
}
printf("Factorial: %ld\n", factorial);
return 0;
}
```